

PROJECT REPORT
at
GURU NANAK DEV ENGINEERING COLLEGE

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE
AWARD OF THE DEGREE OF BACHELOR OF TECHNOLOGY

(Bachelor of Technology)



SUBMITTED BY:

JASDEEP SINGH (2435556)

SUBMITTED TO:

PF. JASPREET KAUR

DEPARTMENT OF INFORMATION TECHNOLOGY
GURU NANAK DEV ENGINEERING COLLEGE -LUDHIANA
(An Autonomous College Under UGC ACT)

1. Introduction

Social media and microblogging platforms have become the cornerstone of modern digital communication, handling millions of short-text transmissions globally every second. In software engineering education, simulating the structural core of these platforms—such as managing user profiles, tracking chronological post streams, and enforcing platform constraints—offers an invaluable model for understanding system architecture. However, as applications scale from simple educational scripts to enterprise solutions, managing state and data structures in a console-based terminal interface becomes a significant technical challenge. For years, traditional student programming projects have relied on flat, monolithic structures that store data globally and perform calculations directly inside the user interface loop, leading to fragile applications that are prone to data corruption and difficult to scale.

The primary issue with these naive implementation approaches is the complete lack of a decoupled ecosystem. When domain data, validation rules, and scanner-based terminal outputs are housed within the same code block, it creates a tight coupling that violates the Single Responsibility Principle. For example, returning mutable data structures to the user interface allows external callers to manipulate internal state directly, bypassing key business rules such as tweet character limits and user post quotas. Additionally, managing error states using return-codes rather than a structured exception flow makes the codebase verbose, hard to trace, and highly vulnerable to unhandled runtime errors. To address these systemic design flaws, there is a clear need for a refactored application architecture that isolates and automates these core data controls using robust design patterns.

The Twitter Simulator CLI was developed as a comprehensive solution to these architectural inefficiencies. By leveraging Java and adhering strictly to the four core pillars of Object-Oriented Programming (OOP)—Encapsulation, Abstraction, Inheritance, and Polymorphism—the project demonstrates a secure, multi-layered environment that separates terminal menu operations from underlying business logic. The system encapsulates user state, exposes read-only collection views, utilizes a custom exception hierarchy for validations, and couples components loosely through service interfaces. Ultimately, the project demonstrates how an object-oriented codebase can eliminate administrative overhead, secure data state, and build a scalable foundation that can easily adapt to future integrations like graphical user interfaces or database backends.

2. Objectives

The primary goal of this project is to design and implement a robust, object-oriented console application that simulates microblogging operations while practicing clean software engineering patterns. The specific objectives include:

- **Encapsulation of User State:** To protect internal mutable lists containing user tweets from representation exposure, ensuring that data additions can only occur via controlled, validated methods that enforce platform quotas.
- **Separation of Concerns:** To decouple terminal presentation operations (console scanning and menu writing) from core domain services (user profiles, tweet validations, and storage structures), allowing either component to evolve independently.
- **Type Abstraction:** To establish clean interfaces (Post and TwitterService) defining standard behaviors, making the overall codebase loosely coupled, flexible, and easily adaptable to database or graphical extensions.
- **Hierarchical Modeling:** To leverage inheritance by creating base abstract classes that handle common behavior (such as automatic timestamp generation) while using child classes to model specific tweet validation rules.
- **Reliable Error Handling:** To implement a custom checked exception hierarchy (TwitterException and its subclasses) to manage data anomalies (like exceeding character bounds or posting quotas) gracefully without causing program crashes.
- **Performance and Efficiency:** To construct optimized, in-memory storage architectures using Java Collections (HashMap for $O(1)$ fast lookups and ArrayList for ordered chronological posts) to ensure immediate feed query response.

3. System Requirements

Hardware Requirements

- **Processor:** Intel Core i3 or above
- **RAM:** 4 GB minimum
- **Hard Disk:** 20 GB free space
- Keyboard and Monitor

Software Requirements

- **Operating System:** Linux (Ubuntu/Debian/Fedora) / Windows 10/11 / macOS
- **Programming Language:** Java
- **IDE:** VS Code / IntelliJ IDEA / NetBeans / Eclipse
- **JDK Version:** JDK 17 or above

4. Methodology

The development of the Twitter Simulator CLI followed a structured, object-oriented software development lifecycle, focusing on modularity, data security, encapsulation, and type safety. The implementation methodology is divided into five key phases:

4.1 System Architecture

The application is built on a decoupled, multi-layered architecture to ensure a clean separation of concerns:

Presentation Layer (TwitterApp): Responsible for managing terminal-oriented menu inputs via the Java Scanner utility, displaying lists, and outputting validation results to the user.

Service Layer (TwitterService and InMemoryTwitterService): Acts as the business logic coordinator. It validates execution rules, coordinates user lookups, and registers tweets, shielding the data layer from the UI.

Domain Data Layer (TwitterUser, Post, Tweet): Houses the raw entity states and implements structural construction checks.

4.2 Class Model and Inheritance Design

The domain models were developed using a hierarchical architecture that maximizes code reuse and ensures data consistency:

The post interface defines standard getters for contents and timestamps.

An abstract class base post implements this interface, centralizing the chronological timestamp generation logic using the Java Time API.

The tweet class extends base post, validating the length constraint of 60 characters upon object creation.

4.3 Service Integration and API Design

The communication between the presentation controller and the backend is mediated by interfaces. The twitter service interface defines two public entry points (routes):

postTweet(String username, String content): Performs validation checks and creates tweets for the target user.

getTweets(String username): Fetches a read-only list of tweets for a specific username.

By utilizing constructor dependency injection, the CLI interface depends entirely on this interface rather than a concrete implementation class, preventing tight coupling.

4.5 Security and Access Control (Data Shielding)

Since the program does not use a remote network layer, security was implemented at the compiler and memory levels:

Representation Exposure Protection: The `getPosts()` method in the user class returns an unmodifiable list wrapper. This ensures that external code cannot bypass validation rules (e.g., trying to call `clear()` or `add()` directly on the list).

Immutable State Attributes: Core fields like `username`, `content`, and `timestamp` are declared `final`, protecting the data from post-creation modifications.

Exception Safety Routing: Domain validation failures (such as exceeding quotas or character limits) throw specific checked exception types (e.g., `TweetLimitExceededException`), which are propagated to the presentation layer where they are handled cleanly.

5. OOPs Concept Used

The application was designed specifically to serve as a showcase for the four primary pillars of Object-Oriented Programming (OOP). Below is a detailed breakdown of how each concept is realized in the code:

5.1 Encapsulation

Encapsulation involves binding data and methods together into a single unit (a class) while restricting direct access to the object's internals to protect its state. In this project:

- **Private Class Fields:** In the user class, fields like username and posts are declared private and final. They cannot be accessed or altered directly from outside the class.
- **Representation Exposure Protection:** The `getPosts()` method does not return a direct reference to the internal list of posts. Instead, it returns a read-only wrapper using `Collections.unmodifiableList(posts)`. This ensures that calling methods like `clear()` or `add()` from the user interface layer will throw an exception, preserving the internal limit constraints.
- **Controlled Mutations:** The only way to add posts is via the `addPost(Post post)` method, which executes internal quota checks before modifying the list.

5.2 Abstraction

Abstraction focuses on hiding complex implementation details and showing only the essential features of an object. This simplifies interaction and decouples components:

- **Post Interface:** The presentation layer only interacts with the Post interface. It does not need to know how the timestamp is generated or formatted; it simply calls `getTimestamp()` and `getContent()`.
- **Service Interface:** The TwitterApp CLI menu does not interact directly with user classes or storage maps. It depends on the TwitterService interface contract, which exposes clean business routes (`postTweet` and `getTweets`) while hiding all memory storage mechanisms.

5.3 Inheritance

Inheritance allows a class to acquire the properties and methods of another class, promoting code reusability and hierarchical organization:

- **Class Hierarchy:** The abstract base post class handles general post attributes (storing text and generating timestamps). The concrete tweet class extends

the base post class, inheriting these fields and adding specific validation checks for character limits.

- **Exception Hierarchy:** A custom checked exception class (TwitterException) serves as the base class for specific exceptions like UserNotFoundException, TweetLimitExceededException, and InvalidTweetException, organizing runtime error structures hierarchically.

5.4 Polymorphism

Polymorphism allows objects to take on multiple forms or behave differently based on the context, using dynamic binding at runtime:

- **Interface Implementation:** The InMemoryTwitterService class implements the TwitterService interface. At runtime, the application wires the interface reference to the concrete implementation, meaning the UI layer could easily bind to a DatabaseTwitterService implementation in the future without changing its menu code.
- **Dynamic Post Processing:** The application processes all list results as type Post rather than Tweet, meaning new post sub-types can be introduced dynamically at runtime.

6. Result

The project successfully resulted in a fully functional, console-based Twitter Simulator application that adheres to all major object-oriented programming paradigms. The platform achieved its goal of simulating microblogging operations, providing a clean separation between user interaction and state storage.

5.1 Functional Outcomes

The final implementation provides the following results:

- **Dynamic Profile Provisioning:** The application dynamically handles user registrations on demand when a tweet is posted for a new username, creating a new user profile automatically.
- **Real-Time Chronological Feed:** Users can retrieve a chronological list of tweets for any registered user, with each post displaying a precise system-generated timestamp showing when it was authored.
- **Length-Constraint Validation:** The system successfully intercepts tweets that exceed 60 characters, rejecting the transaction and printing a descriptive error message to the terminal without stopping program execution.

- **Quota Limit Validation:** The system effectively tracks user tweet counts, blocking additional tweets once a user reaches the maximum limit of 10 posts and displaying a clear limit-exceeded warning.

5.2 Observations

- **Strict Data Encapsulation:** The implementation of read-only collection wrappers ensured that the internal tweet storage list could not be altered from outside the user class, preventing external state corruption.
- **Decoupled Architecture:** Due to the separation of user interface logic and service contracts, the core business engine runs independently from terminal console controls, making the application highly maintainable.
- **Robust Error Propagation:** The use of a custom checked exception hierarchy allowed the application to bubble up validation failures cleanly from the deep validation checks to the user menu loop.
- **Memory Performance:** By utilizing optimized hash maps and list structures, the platform performs lookup and insertion operations in near-instantaneous times with minimal memory footprint.

```

[user@dev-machine ~]$ java TwitterSimulation
1. Add Tweet
2. Get Tweets of a User
3. Exit
Enter choice: 1
Enter username: john_doe
Enter tweet (max 60 characters): Hello from the OOP Refactor!
Tweet added successfully!

1. Add Tweet
2. Get Tweets of a User
3. Exit
Enter choice: 2
Enter username to get tweets: john_doe
Tweets of john_doe:
[2026-05-24 17:19:51] Hello from the OOP Refactor!

1. Add Tweet
2. Get Tweets of a User
3. Exit
Enter choice: 3
Exiting...

```

7. Conclusion

The development of the Twitter Simulator CLI has successfully demonstrated how core software engineering concepts can be implemented in a console-based environment to build clean, maintainable, and type-safe systems. Transitioning from a basic scripting approach to a layered, modular framework has shown that even small-scale tools benefit from structured planning. The application successfully fulfills its target requirements by managing user accounts dynamically, tracking chronological post streams, and enforcing platform constraints like character and quota limits, all while maintaining excellent runtime execution.

Through this architectural design, the benefits of the four pillars of Object-Oriented Programming (OOP) were realized in practice. The encapsulation of user variables and read-only collection wrappers completely shields the internal state from representation exposure. Abstraction and polymorphism decoupling through services ensure that the user interface operates independently of memory storage logic. Additionally, the custom exception hierarchy handles validation errors gracefully, providing a structured way to handle anomalies without relying on unsafe program exits or return flags.

Overall, this project provided an invaluable learning experience in building robust Java applications and implementing clean-code design patterns. Designing and refactoring this simulator has deepened our understanding of class hierarchies, constructor dependency injections, and compiler-level security. By establishing a solid object-oriented foundation, the simulator stands ready for future advancements, such as migrating to a relational database persistence layer or wrapping the core service engine in a modern graphical user interface.